

Two-Variable Models

Ray Chandonnet and Barrett Viator

2025-08-13

```
# Import Libraries
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.0      v stringr    1.5.1
```

```
## v ggplot2    3.5.2      v tibble     3.3.0
```

```
## v lubridate  1.9.4      v tidyr      1.3.1
```

```
## v purrr      1.1.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(readxl)
```

```
library(writexl)
```

```
library(skimr)
```

```
library(knitr)
```

```
library(kableExtra)
```

```
##
```

```
## Attaching package: 'kableExtra'
```

```
##
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      group_rows
```

```
library(tidyr)
```

```
library(ggpmisc)
```

```
## Loading required package: ggpp
```

```
## Registered S3 methods overwritten by 'ggpp':
```

```
##      method      from
```

```
## heightDetails.titleGrob ggplot2
```

```
## widthDetails.titleGrob  ggplot2
```

```
##
```

```
## Attaching package: 'ggpp'
```

```
##
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##      annotate
```

```
library(forcats)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(naniar)
```

```
##
## Attaching package: 'naniar'
##
## The following object is masked from 'package:skimr':
##
##     n_complete
```

```
library(reshape2)
```

```
##
## Attaching package: 'reshape2'
##
## The following object is masked from 'package:tidyr':
##
##     smiths
```

```
library(scales)
```

```
##
## Attaching package: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

```
library(stringr)
library(recipes)
```

```
##
## Attaching package: 'recipes'
##
## The following object is masked from 'package:stringr':
##
##     fixed
##
## The following object is masked from 'package:stats':
##
##     step
```

```
library(broom)
library(here)
```

```
## here() starts at /Users/raychandonnet/Chandonnet Family Dropbox/Chandonnet Family Folder/Ray Personal
```

```
library(rlang)
```

```
##
```

```

## Attaching package: 'rlang'
##
## The following objects are masked from 'package:purrr':
##
##     %%, flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##     flatten_raw, invoke, splice
library(rsample)
library(xts)

## Loading required package: zoo
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
##
## ##### Warning from 'xts' package #####
## #
## # The dplyr lag() function breaks how base R's lag() function is supposed to #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
## # source() into this session won't work correctly. #
## #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
## # dplyr from breaking base R's lag() function. #
## #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning. #
## #
## #####
##
## Attaching package: 'xts'
##
## The following objects are masked from 'package:dplyr':
##
##     first, last
library(randomForest)

## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin

```

```

library(caTools)
library(caret)

## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:rsample':
##
##   calibration
##
## The following object is masked from 'package:purrr':
##
##   lift

library(RANN)
library(xgboost)

##
## Attaching package: 'xgboost'
##
## The following object is masked from 'package:dplyr':
##
##   slice

library(Matrix)

##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##   expand, pack, unpack

library(e1071)

##
## Attaching package: 'e1071'
##
## The following object is masked from 'package:rsample':
##
##   permutations

library(parsnip)

##
## Attaching package: 'parsnip'
##
## The following object is masked from 'package:e1071':
##
##   tune

library(tidymodels)

## -- Attaching packages ----- tidymodels 1.3.0 --
## v dials      1.4.1      v workflows  1.2.0
## v infer      1.0.9      v workflowsets 1.1.1

```

```
## v modeldata      1.5.0      v yardstick      1.3.2
## v tune           1.3.0
## -- Conflicts ----- tidymodels_conflicts() --
## x rlang::%@%()      masks purrr::%@%()
## x ggpp::annotate()  masks ggplot2::annotate()
## x caret::calibration() masks rsample::calibration()
## x randomForest::combine() masks dplyr::combine()
## x scales::discard() masks purrr::discard()
## x Matrix::expand()  masks tidyr::expand()
## x dplyr::filter()   masks stats::filter()
## x xts::first()      masks dplyr::first()
## x recipes::fixed()  masks stringr::fixed()
## x rlang::flatten()  masks purrr::flatten()
## x rlang::flatten_chr() masks purrr::flatten_chr()
## x rlang::flatten_dbl() masks purrr::flatten_dbl()
## x rlang::flatten_int() masks purrr::flatten_int()
## x rlang::flatten_lgl() masks purrr::flatten_lgl()
## x rlang::flatten_raw() masks purrr::flatten_raw()
## x kableExtra::group_rows() masks dplyr::group_rows()
## x rlang::invoke()   masks purrr::invoke()
## x dplyr::lag()       masks stats::lag()
## x xts::last()        masks dplyr::last()
## x caret::lift()     masks purrr::lift()
## x randomForest::margin() masks ggplot2::margin()
## x Matrix::pack()     masks tidyr::pack()
## x e1071::permutations() masks rsample::permutations()
## x yardstick::precision() masks caret::precision()
## x yardstick::recall()  masks caret::recall()
## x yardstick::sensitivity() masks caret::sensitivity()
## x xgboost::slice()     masks dplyr::slice()
## x yardstick::spec()    masks readr::spec()
## x yardstick::specificity() masks caret::specificity()
## x rlang::splice()      masks purrr::splice()
## x recipes::step()      masks stats::step()
## x tune::tune()         masks parsnip::tune(), e1071::tune()
## x Matrix::unpack()     masks tidyr::unpack()
## x Matrix::update()     masks recipes::update(), stats::update()
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##      select

print_regressions <- function(predictions, change = FALSE){
  chart_title <- "Actual vs Predicted - "
  if (change) {
    results_lm <- predictions %>%
      transmute(Actual = YearlyChgHDI, Predicted = .pred)
    chart_title <- paste0(chart_title, "Y-o-Y Change in HDI Index")
    low_bound <- -0.12
    high_bound <- 0.12
  }
}
```

```

} else {
  results_lm <- predictions %>%
    transmute(Actual = HDI_Index, Predicted = .pred)
  chart_title <- paste0(chart_title, "HDI Index")
  low_bound <- 0.3
  high_bound <- 1
}

p_scatter <- ggplot(results_lm, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, color = "red",
             linetype = "dashed", linewidth = 1) +
  coord_equal(xlim = c(low_bound, high_bound), ylim = c(low_bound, high_bound)) +
  labs(
    title = chart_title,
    x = "Actual",
    y = "Predicted"
  ) +
  theme_minimal()
print(p_scatter)
cat('\n\nnewline')

# Residuals histogram
results_lm$Residuals <- results_lm$Actual - results_lm$Predicted
p_resid <- ggplot(results_lm, aes(x = Residuals)) +
  geom_histogram(bins = 30, fill = "lightblue", color = "black") +
  geom_vline(xintercept = 0, color = "red", linetype = "dashed", linewidth = 1) +
  labs(
    title = "Residuals Distribution",
    x = "Residual (Actual - Predicted)",
    y = "Count"
  ) +
  theme_minimal()
print(p_resid)
}

# Read in the saved training and test data sets
train_data <- readRDS(here::here("Data", "train_data.rds"))
test_data <- readRDS(here::here("Data", "test_data.rds"))

# Start with basic regression
model_columns <- c(
  "HDI_Index", "InternetUsersPct"
)

# Filter the training and test data to include only the specified columns
train_filtered <- train_data %>% dplyr::select(all_of(model_columns))
test_filtered <- test_data %>% dplyr::select(all_of(model_columns))

# Use na.omit() to remove rows with any NA values

```

```

train_filtered <- na.omit(train_filtered)
test_filtered <- na.omit(test_filtered)

# libs

set.seed(123)

myrecipe <- recipe(HDI_Index ~ InternetUsersPct, data = train_filtered)

mymodel <- linear_reg() %>%
  set_engine("lm")

myworkflow <- workflow() %>%
  add_recipe(myrecipe) %>%
  add_model(mymodel)

fitted_model <- fit(myworkflow, data = train_filtered)

predictions <- predict(fitted_model, new_data = test_filtered) %>%
  bind_cols(test_filtered)

metrics <- metric_set(rmse, mae, rsq)
metrics(predictions, truth = HDI_Index, estimate = .pred)

```

A tibble: 3 x 3

```

.metric .estimator .estimate 1 rmse standard 0.0955 2 mae standard 0.0770 3 rsq standard 0.634
# (optional) coefficients + summary
tidy(extract_fit_parsnip(fitted_model))

```

A tibble: 2 x 5

```

term estimate std.error statistic p.value 1 (Intercept) 0.467 0.00357 131. 0 2 InternetUsersPct 0.0394 0.000530
74.3 0
glance(extract_fit_parsnip(fitted_model))

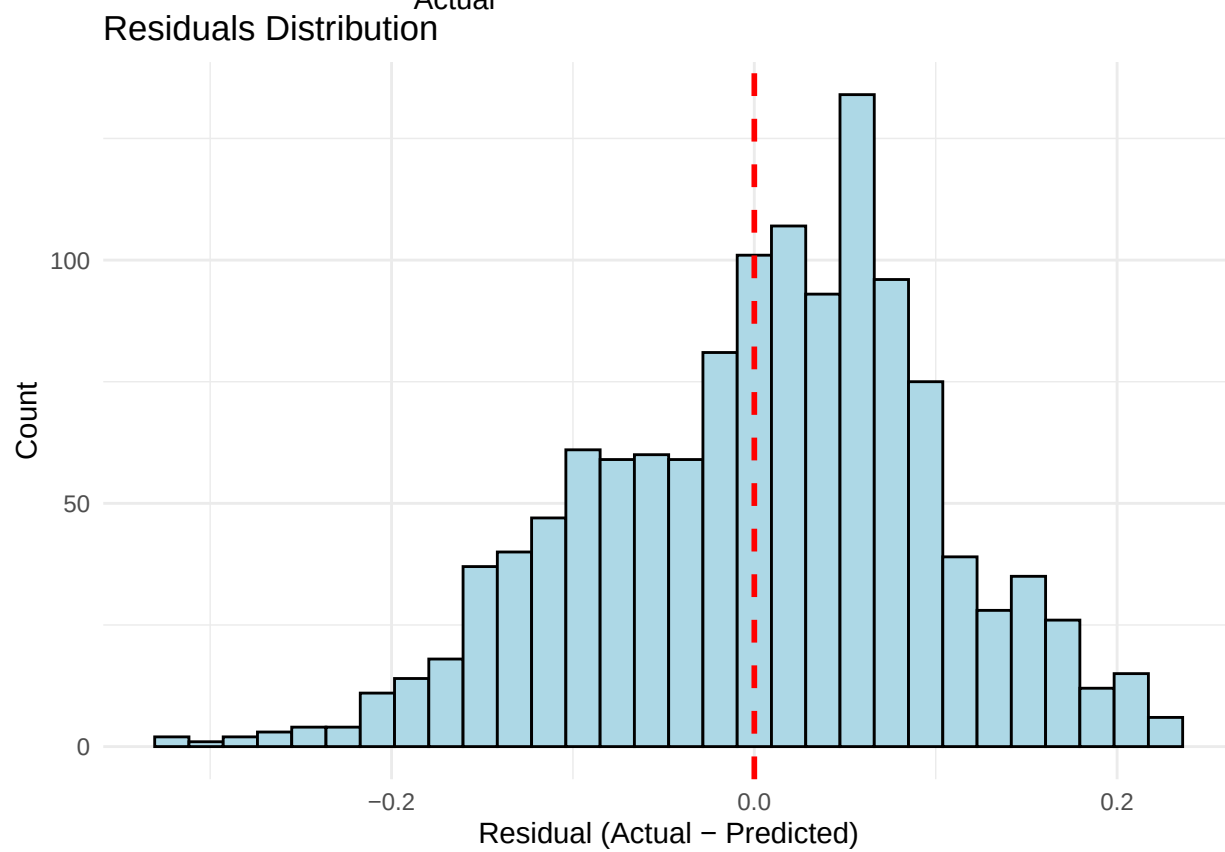
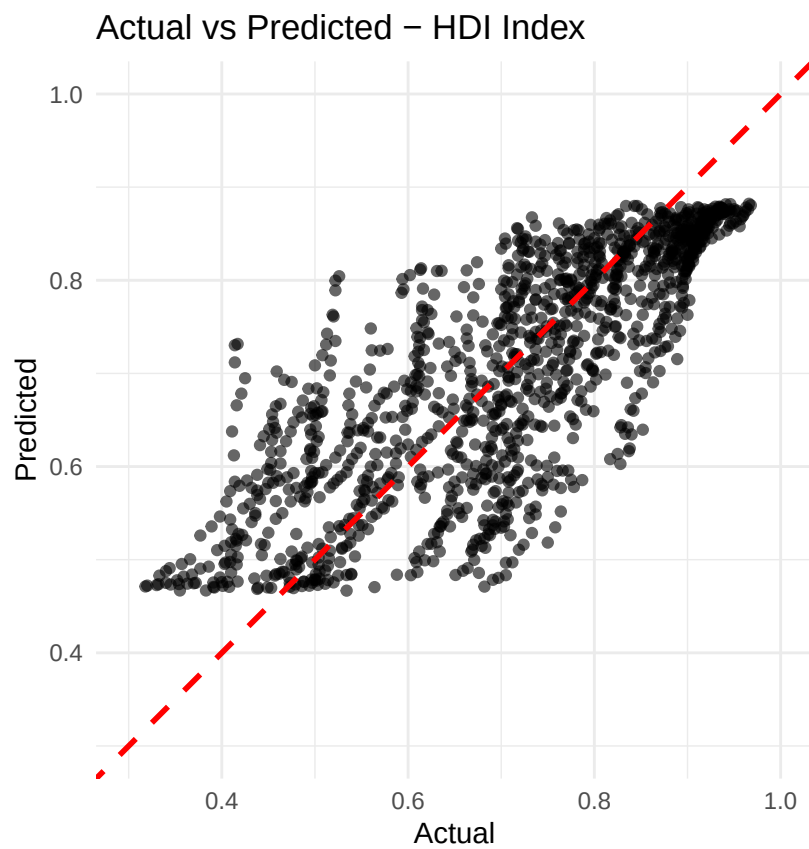
```

A tibble: 1 x 12

```

r.squared adj.r.squared sigma statistic p.value df logLik AIC BIC 1 0.661 0.660 0.0932 5524. 0 1 2712. -5419.
-5401. # i 3 more variables: deviance , df.residual , nobs
print_regressions(predictions)

```




```

# Let's try a quadratic regression, see if that's better:
set.seed(123)

myrecipe <- recipe(HDI_Index ~ InternetUsersPct, data = train_filtered) %>%
  step_poly(InternetUsersPct, degree=2)

mymodel <- linear_reg() %>%
  set_engine("lm")

myworkflow <- workflow() %>%
  add_recipe(myrecipe) %>%
  add_model(mymodel)

fitted_model <- fit(myworkflow, data = train_filtered)

predictions <- predict(fitted_model, new_data = test_filtered) %>%
  bind_cols(test_filtered)

metrics <- metric_set(rmse, mae, rsq)
metrics(predictions, truth = HDI_Index, estimate = .pred)

```

A tibble: 3 x 3

```
.metric .estimator .estimate 1 rmse standard 0.0954 2 mae standard 0.0769 3 rsq standard 0.635
```

```

# (optional) coefficients + summary
tidy(extract_fit_parsnip(fitted_model))

```

A tibble: 3 x 5

```

term estimate std.error statistic p.value 1 (Intercept) 0.698 0.00175 399. 0
2 InternetUsersPct_poly_1 6.92 0.0932 74.3 0
3 InternetUsersPct_poly_2 0.0671 0.0932 0.720 0.472

```

```
glance(extract_fit_parsnip(fitted_model))
```

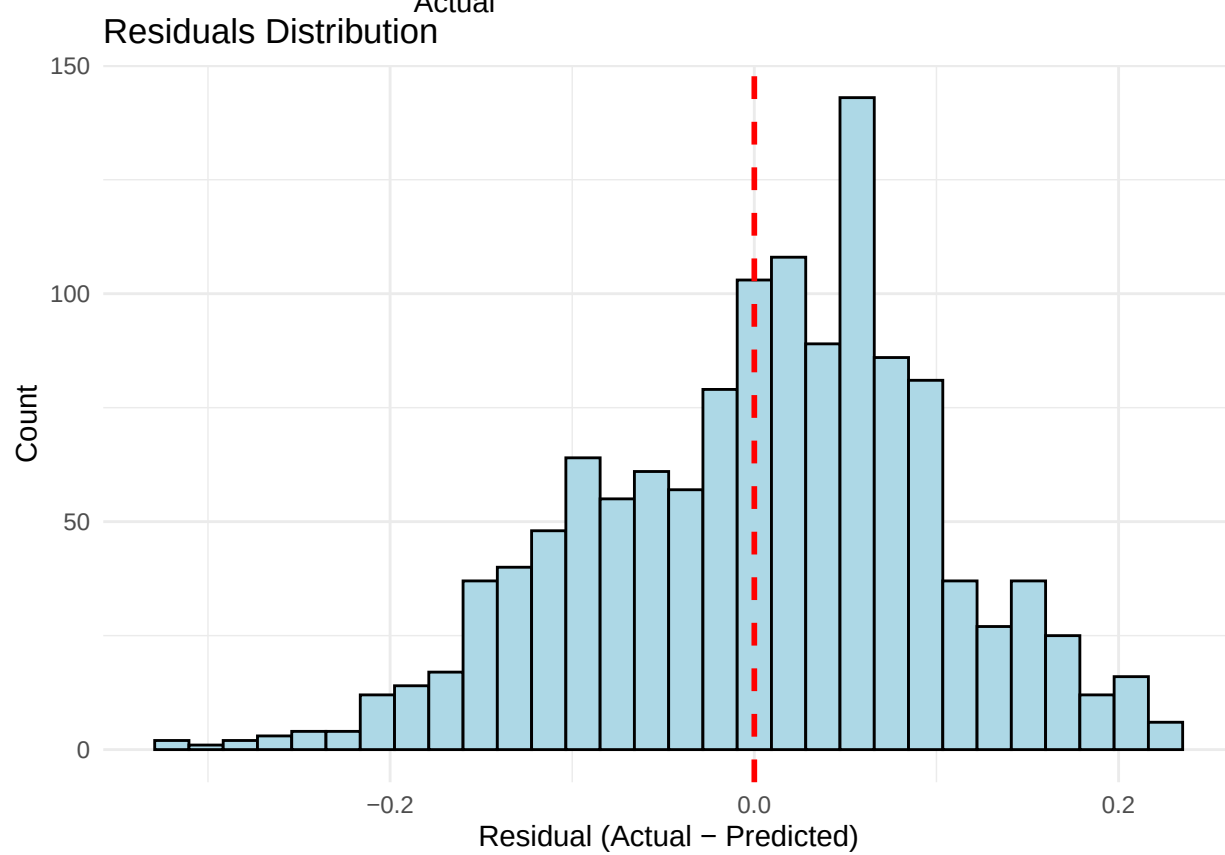
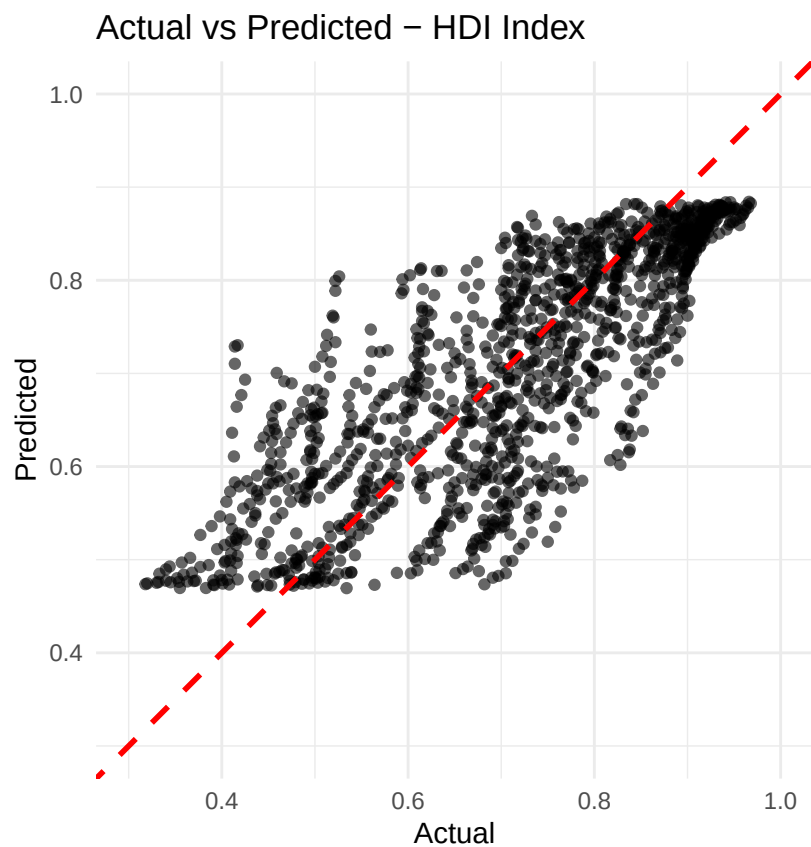
A tibble: 1 x 12

```

r.squared adj.r.squared sigma statistic p.value df logLik AIC BIC 1 0.661 0.660 0.0932 2762. 0 2 2713. -5417.
-5394. # i 3 more variables: deviance , df.residual , nobs

```

```
print_regressions(predictions)
```



```
#####
#
# Predicting HDI CATEGORY

# Use the category outcome instead of the index
model_columns <- c("HDI_Category", "InternetUsersPct")

train_filtered <- train_data %>% dplyr::select(all_of(model_columns)) %>% drop_na()
test_filtered <- test_data %>% dplyr::select(all_of(model_columns)) %>% drop_na()

long <- c("Low human development", "Medium human development",
          "High human development", "Very high human development")
short <- c("Low", "Medium", "High", "Very High")

train_filtered$HDI_Category <- factor(train_filtered$HDI_Category, levels = long,
                                     labels = short, ordered=TRUE)
test_filtered$HDI_Category <- factor(test_filtered$HDI_Category, levels = long,
                                    labels = short, ordered=TRUE)

set.seed(123)

# Recipe: category ~ InternetUsersPct
myrecipe <- recipe(HDI_Category ~ InternetUsersPct, data = train_filtered)

# Model: multinomial logistic regression
mymodel <- multinom_reg() %>%
  set_engine("nnet") %>%
  set_mode("classification")

myworkflow <- workflow() %>%
  add_recipe(myrecipe) %>%
  add_model(mymodel)

fitted_model <- fit(myworkflow, data = train_filtered)

# Predictions (class + probabilities)
predictions <- augment(fitted_model, new_data = test_filtered)

# Metric: accuracy (simple, minimal change)
accuracy(predictions, truth = HDI_Category, estimate = .pred_class)
```

A tibble: 1 x 3

```
.metric .estimator .estimate 1 accuracy multiclass 0.564
```

```
# Visualize model performance
```

```
confusion_matrix <- table(Actual = predictions$HDI_Category,
                          Predicted = predictions$.pred_class)
print("Confusion Matrix:")
```

```
[1] "Confusion Matrix:"
```

```
print(confusion_matrix)
```

Predicted

Actual Low Medium High Very High Low 201 67 22 3 Medium 97 84 71 15 High 42 53 105 119 Very High 0
14 63 343

```
# Show distribution of data by category, shortening the category names
short_cat <- gsub(" human development", "", train_data$HDI_Category)
prop.table(table(short_cat)) * 100
```

```
short_cat High Low Medium Very high 24.87741 23.47172 23.30827 28.34260
```

```
#####
#
# Does using "change" variables help?
#
model_columns <- c(
  "YearlyChgHDI", "YearlyChgInternet"
)

# Filter the training and test data to include only the specified columns

train_filtered <- train_data %>% dplyr::select(all_of(model_columns))
test_filtered <- test_data %>% dplyr::select(all_of(model_columns))

# Use na.omit() to remove rows with any NA values

train_filtered <- na.omit(train_filtered)
test_filtered <- na.omit(test_filtered)

set.seed(123)

myrecipe <- recipe(YearlyChgHDI ~ YearlyChgInternet, data = train_filtered)

mymodel <- linear_reg() %>%
  set_engine("lm")

myworkflow <- workflow() %>%
  add_recipe(myrecipe) %>%
  add_model(mymodel)

fitted_model <- fit(myworkflow, data = train_filtered)

predictions <- predict(fitted_model, new_data = test_filtered) %>%
  bind_cols(test_filtered)

metrics <- metric_set(rmse, mae, rsq)
metrics(predictions, truth = YearlyChgHDI, estimate = .pred)
```

A tibble: 3 x 3

.metric .estimator .estimate 1 rmse standard 0.00672 2 mae standard 0.00425 3 rsq standard 0.000594

```
# (optional) coefficients + summary
tidy(extract_fit_parsnip(fitted_model))
```

A tibble: 2 x 5

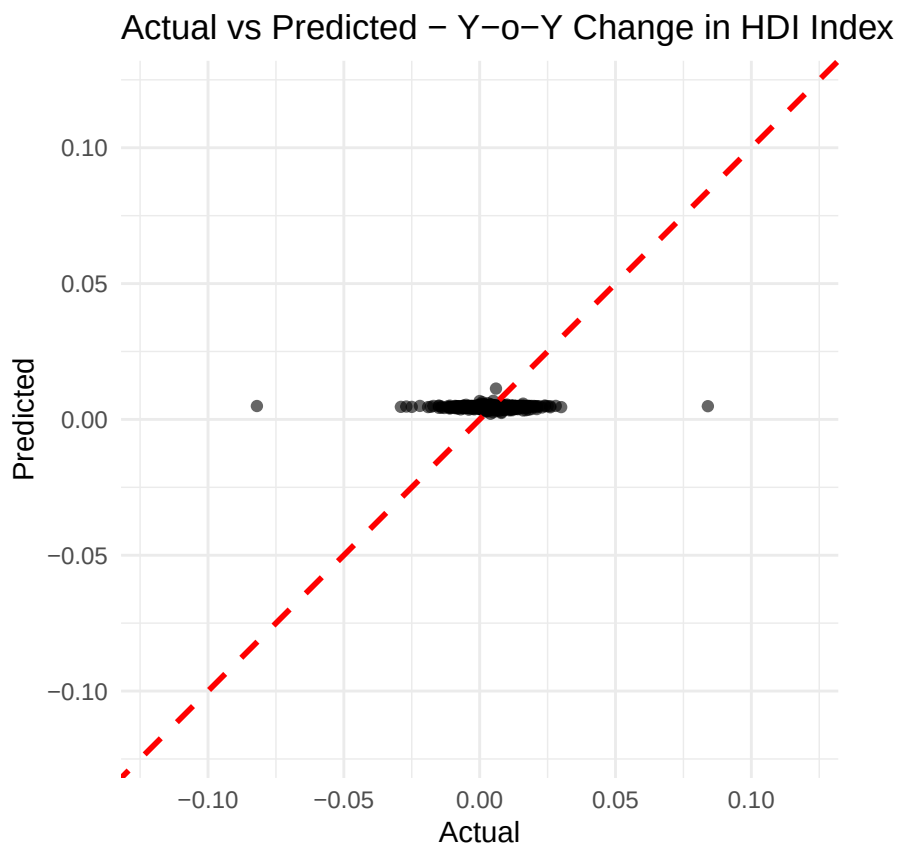
```
term estimate std.error statistic p.value 1 (Intercept) 0.00497 0.000167 29.8 1.02e-169 2 YearlyChgInternet
-0.000175 0.0000514 -3.41 6.57e- 4
```

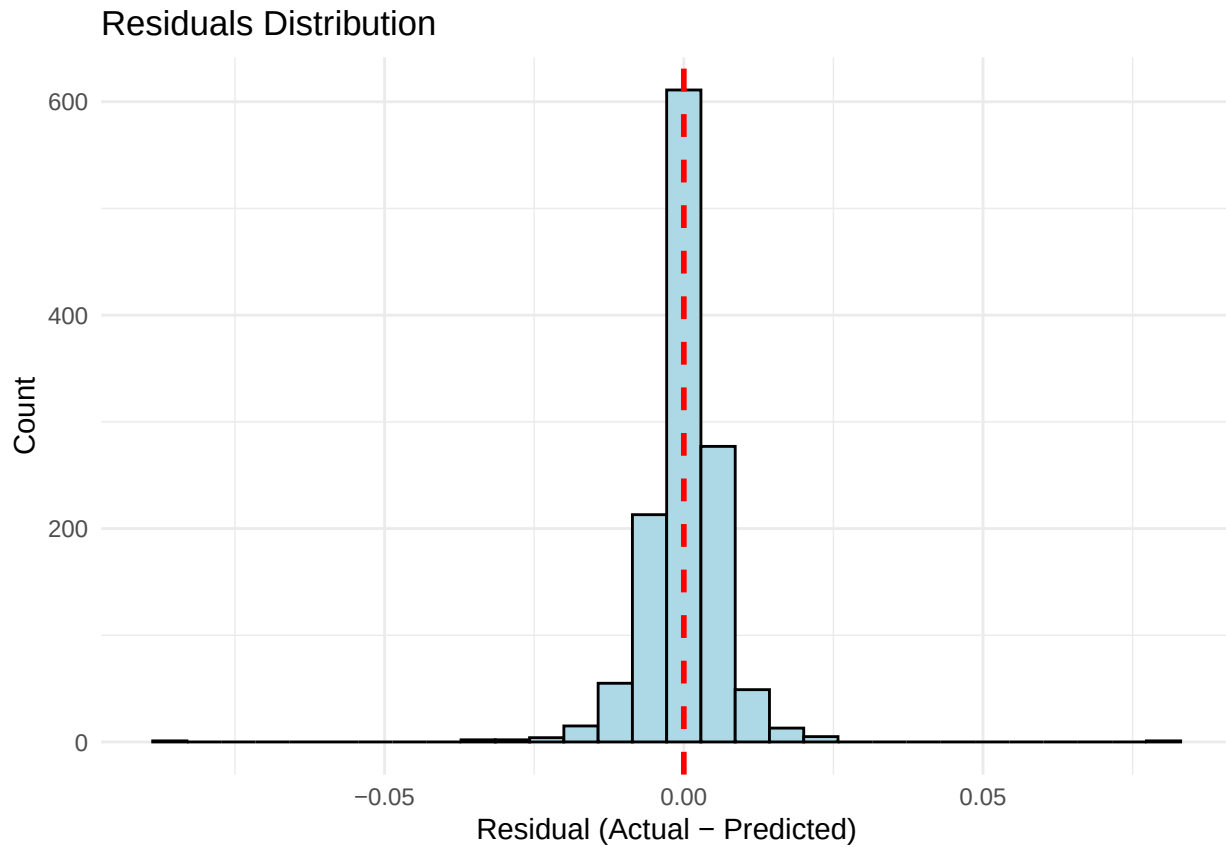
```
glance(extract_fit_parsnip(fitted_model))
```

A tibble: 1 x 12

```
r.squared adj.r.squared sigma statistic p.value df logLik AIC BIC 1 0.00416 0.00380 0.00677 11.6 6.57e-4 1
9971. -19936. -19918. # i 3 more variables: deviance , df.residual , nobs
```

```
print_regressions(predictions, change = TRUE)
```

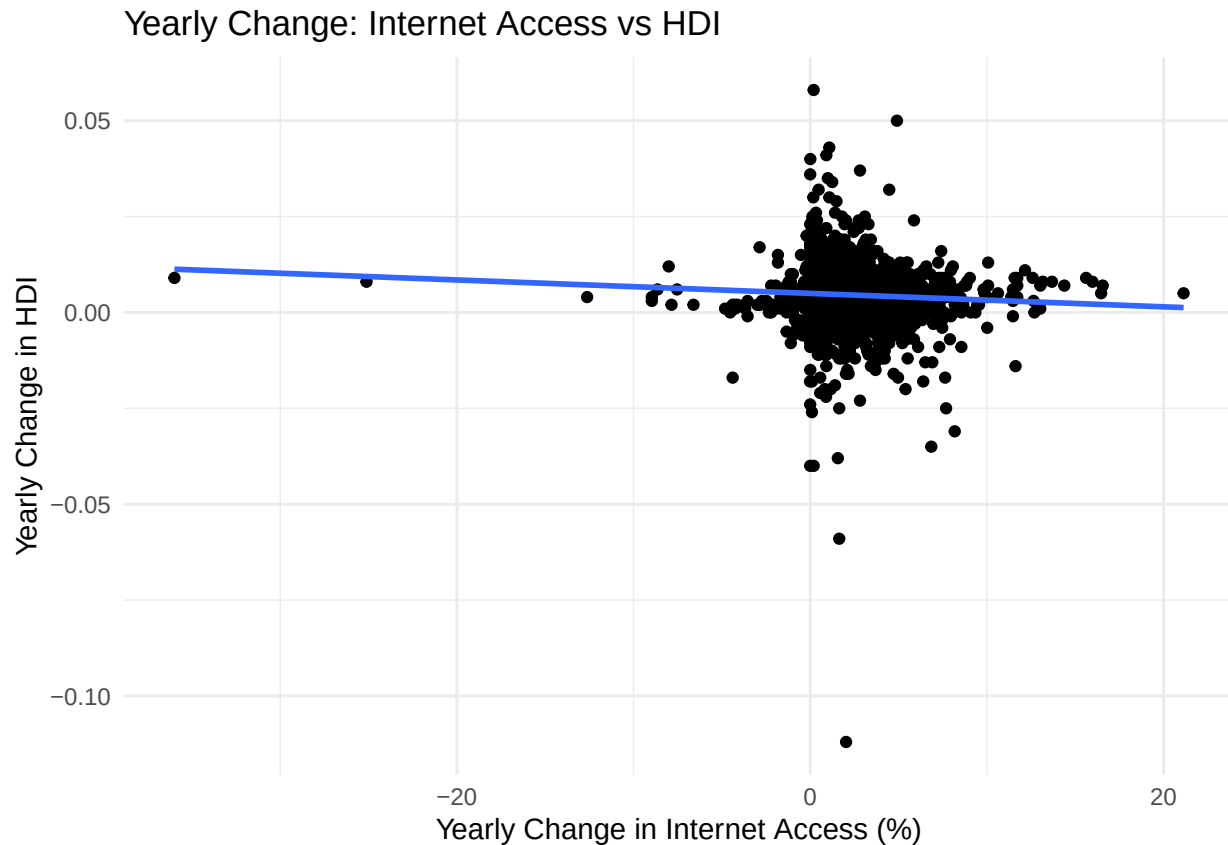




```
library(ggplot2)

ggplot(train_filtered, aes(x = YearlyChgInternet, y = YearlyChgHDI)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)+
  labs(
    title = "Yearly Change: Internet Access vs HDI",
    x = "Yearly Change in Internet Access (%)",
    y = "Yearly Change in HDI"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
#####
#
# How about incorporating a 3 year lag
#
model_columns <- c(
  "HDI_Index", "Cumulative3yrChg_InternetUsersPct"
)

# Filter the training and test data to include only the specified columns

train_filtered <- train_data %>% dplyr::select(all_of(model_columns))
test_filtered <- test_data %>% dplyr::select(all_of(model_columns))

# Use na.omit() to remove rows with any NA values

train_filtered <- na.omit(train_filtered)
test_filtered <- na.omit(test_filtered)

set.seed(123)

myrecipe <- recipe(HDI_Index ~ Cumulative3yrChg_InternetUsersPct, data = train_filtered)

mymodel <- linear_reg() %>%
  set_engine("lm")

myworkflow <- workflow() %>%
  add_recipe(myrecipe) %>%
```

```

add_model(mymodel)

fitted_model <- fit(myworkflow, data = train_filtered)

predictions <- predict(fitted_model, new_data = test_filtered) %>%
  bind_cols(test_filtered)

metrics <- metric_set(rmse, mae, rsq)
print(metrics(predictions, truth = HDI_Index, estimate = .pred))

```

A tibble: 3 x 3

```
.metric .estimator .estimate 1 rmse standard 0.148 2 mae standard 0.120 3 rsq standard 0.121
```

```

# (optional) coefficients + summary
print(tidy(extract_fit_parsnip(fitted_model)))

```

A tibble: 2 x 5

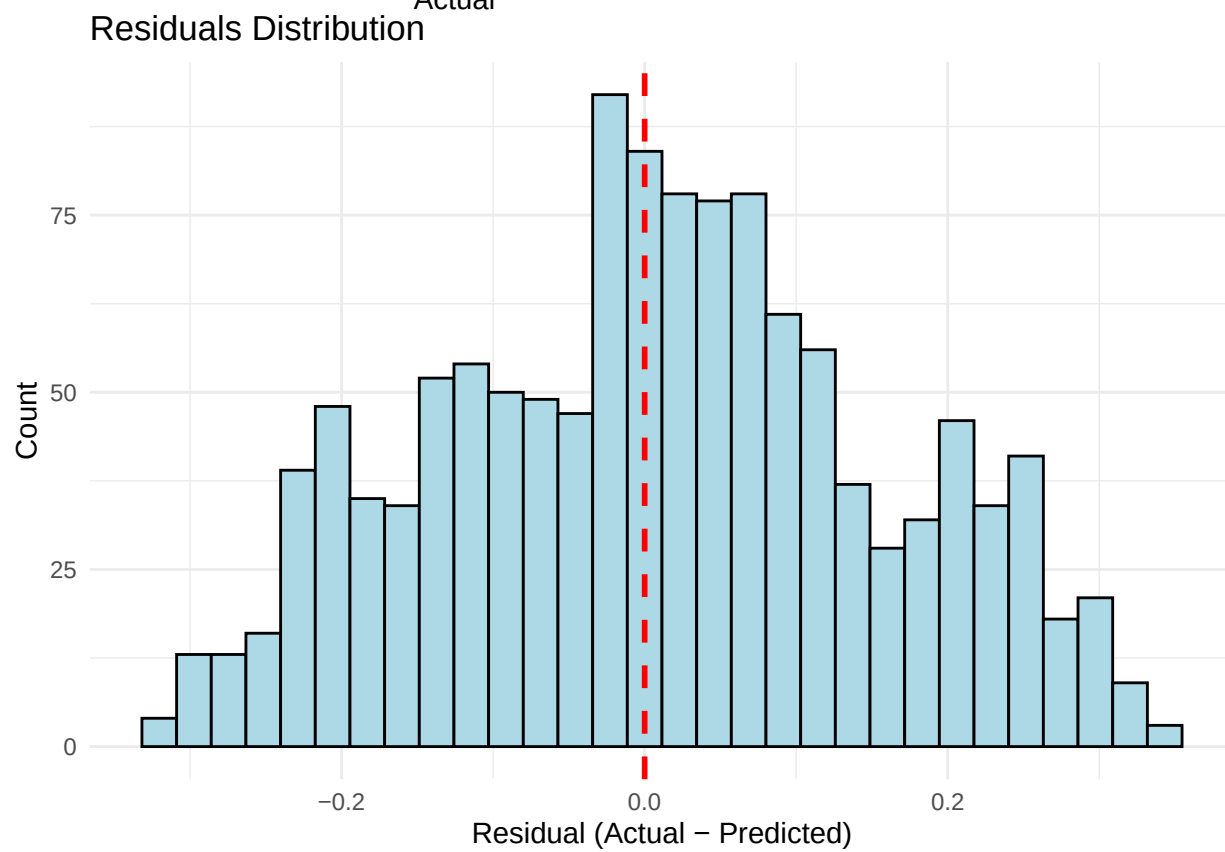
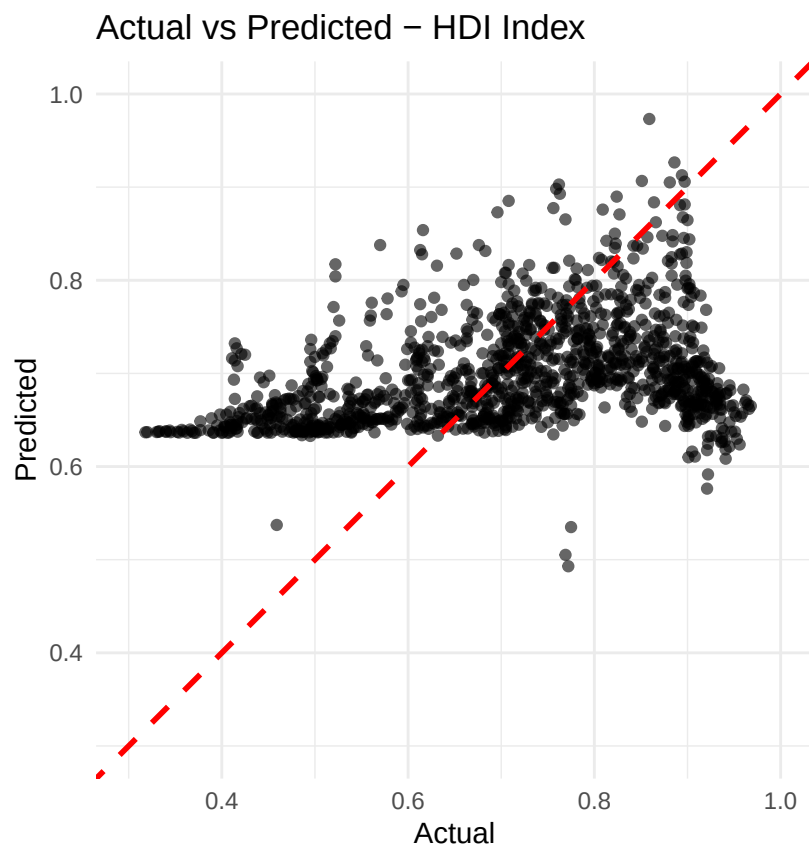
```
term estimate std.error statistic p.value 1 (Intercept) 0.636 0.00414 154. 0
2 Cumulative3yrChg_InternetUsersPct 0.00765 0.000366 20.9 5.58e-90
```

```
print(glance(extract_fit_parsnip(fitted_model)))
```

A tibble: 1 x 12

```
r.squared adj.r.squared sigma statistic p.value df logLik AIC BIC 1 0.136 0.135 0.149 436. 5.58e-90 1 1343.
-2681. -2663. # i 3 more variables: deviance , df.residual , nobs
```

```
print_regressions(predictions, change = FALSE)
```

```
library(ggplot2)

ggplot(train_filtered, aes(x = Cumulative3yrChg_InternetUsersPct, y = HDI_Index)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)+
  labs(
    title = "Yearly Change: Internet Access vs HDI",
    x = "Yearly Change in Internet Access (%)",
    y = "Yearly Change in HDI"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

